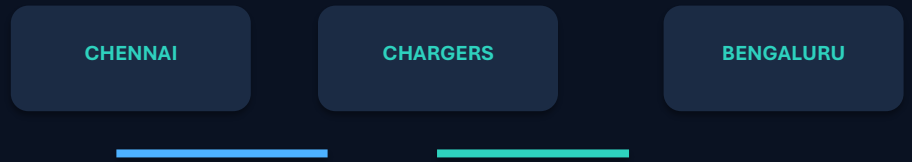
DESIGN AND ANALYSIS OF ALGORITHMS • BATCH 13

EV LOGISTICS: CHARGING STOPS

Finding the minimum-stop route with a greedy strategy



350 km

TOTAL JOURNEY

24I304-Adhvik Pradeep | 24i311-Dharanidharan P | 24I322-Karthik Gopalakrishnan | 24I324-Kavin Vikas S | 24i339-Pranav Karthick K

The decision is simple; the constraint is not

We must reach the destination without exceeding the vehicle’s maximum range between charges.

350 km

Journey length

120 km

Maximum range per full charge

40 min

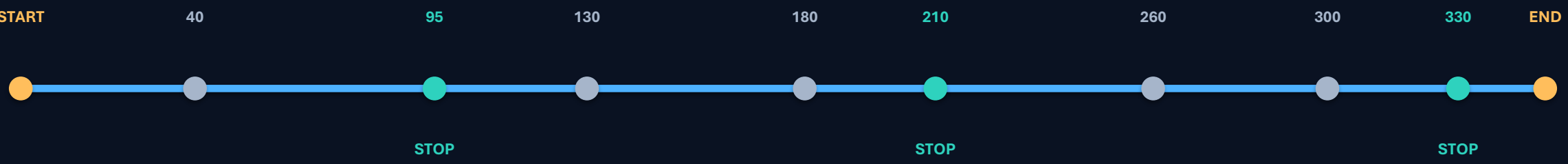
Time penalty per charging stop

Goal: choose the fewest intermediate charging stations so that every travel leg is ≤ 120 km.

Assumption: every selected stop restores the vehicle to full charge; station locations are known.

Translate the road into ordered points

All locations are measured as kilometres from the starting point.



A feasible route is a sequence of points where each consecutive gap is no more than 120 km.

Candidate chargers: 40, 95, 130, 180, 210, 260, 300, 330 km

Why Greedy fits this version of the problem

At every decision point, move as far forward as possible without breaking the range limit.

GREEDY RULE

Choose the farthest charger within 120 km of the current position.

One local choice, repeated.

WHY THIS CHOICE IS SAFE

- It makes the greatest possible progress now.
- It cannot make any later charger farther away.
- The sorted order allows a single forward scan.

The algorithm: scan once, never look back

The index only moves forward; each charger is examined at most once.

```
current = 0; i = 0; stops = []
while destination - current > 120:
    farthest = current
    while i < n and chargers[i] - current <= 120:
        farthest = chargers[i]; i += 1
    if farthest == current: return INFEASIBLE
    stops.append(farthest); current = farthest
return stops
```

KEY DETAIL

The inner scan consumes every reachable charger once. If no charger is reachable and the destination is still too far, the trip is infeasible.

Trace: the farthest reachable charger each time

The selected stop is highlighted in teal at each decision.

Current position	Reach limit	Reachable options	Greedy decision
0 km	Reach up to 120	40, 95	**95 km**
95 km	Reach up to 215	130, 180, 210	**210 km**
210 km	Reach up to 330	260, 300, 330	**330 km**
330 km	Reach up to 450	Destination 350	**Finish**

Route: 0 → 95 → 210 → 330 → 350 km

Final route and stop count

Every leg satisfies the 120 km range constraint.

0 → 95

95 km

95 → 210

115 km

210 → 330

120 km

330 → 350

20 km

3 CHARGING STOPS

120 MINUTES STOP TIME

Why can we trust the greedy answer?

Exchange argument + induction establish optimality.

EXCHANGE ARGUMENT

- Let g be the farthest reachable charger.
- Any feasible first stop o must satisfy o ≤ g.
- Replacing o with g cannot make a later stop harder to reach.
- So some optimal solution starts with g.

INDUCTION

- The remaining route after g is the same type of problem.
- Apply the same exchange to the suffix.
- The greedy route is feasible and uses no more stops than an optimum.

Why this is efficient

The scan is linear after sorting because the pointer never moves backwards.

O(n)

Greedy scan

O(n log n)

Including sorting

O(k)

Output list of k stops

Best case: destination already within range.

Worst case: scan all chargers; each is processed once.

If input is already sorted, no additional sorting is needed.

Data structures: choose for the workload

This is a static, ordered-location problem, so a sorted array/list is the natural fit.

Structure	Search / scan	Update cost	Verdict
Sorted array/list	O(n) total scan	O(n) insert	**BEST FIT**
Linked list	O(n) traversal	O(1) if node known	**No random access**
Balanced BST	O(log n) successor	O(log n)	**Useful for dynamic updates**

Sorted list: cache-friendly, compact and already aligned with the algorithm’s access pattern.

Why not the other design techniques?

They are possible approaches, but they add work or do not exploit the problem's structure.

Divide & Conquer	Boundary states make merging awkward; a single scan is simpler.
Dynamic Programming	Valid $O(n^2)$ solution; unnecessary when greedy dominance gives $O(n)$.
Backtracking	May enumerate up to 2^n charger subsets.
Branch & Bound	Can prune, but worst-case search remains exponential.

Important: DP is not incorrect here; it is simply less efficient for this special case.

What changes when the assumptions change?

The greedy rule is optimal only under the fixed, equal-range model.

A • DIFFERENT RANGE AT EACH STATION

- Each station i has its own range R_i.
- Create edge i → j if $x_j - x_i \le R_i$.
- Run BFS on the forward graph to minimize stops.

B • DIFFERENT PRICES / PARTIAL CHARGING

- Minimum stops no longer means minimum cost.
- Track charger and state-of-charge as a state.
- Use shortest path / dynamic programming.

Cost counterexample: farthest is not always cheapest

With different electricity prices, the stop-minimizing greedy rule can lose on cost.

EXAMPLE INPUT

Battery range: 120 km

Stations: 60, 110, 180 km

Prices: ₹1, ₹10, ₹1 per range-unit

Destination: 240 km

FARTHEST-FIRST: ₹760

At 110 km, buy 70 units at ₹10; then 60 at ₹1.

CHEAPER PLAN: ₹120

At 60 km, buy 60 at ₹1; at 180, buy 60 at ₹1.

Lesson: optimize the actual objective—cost—not a proxy such as maximum forward distance.

From classroom model to real EV route planning

Real route planners account for far more than a fixed range and fixed station map.

OUR SIMPLIFIED MODEL

- Straight-line distance
- Fixed 120 km range
- Known charger positions
- Equal 40-minute stop time

REAL-WORLD ROUTE PLANNERS

- Vehicle-specific energy consumption
- Elevation, weather and speed
- Live charger availability and power
- Charging curves, dwell time and cost

Example reference: A Better Routeplanner (ABRP) • abetterrouteplanner.com

The answer—and the conditions behind it

Greedy is the right tool for this fixed-range, minimum-stop version of the problem.

0 → 95 → 210 → 330 → 350 km

3	120 min	O(n)
minimum stops	stop time	scan time

Thank you